PROG6212·POEPART1·SECTIONB

# RaceDay API

Endpoint Plan - a role-based REST API for organising and entering running, walking, and cycling events

**Two Roles. One App.**
**Organiser** - creates events, defines race categories, reviews enrollments, and captures results.
**Participant** - browses events, enrolls into a category, records payment attempts, and checks their own results.

**Colour legend for HTTP methods:** GET read POST create PUT update DELETE remove

**Alignment assumptions**
- Authentication credentials are handled by the API authentication layer; the Section A ERD and Section C schema model the RaceDay domain entities.
- After authentication, the API uses the authenticated user's unique email and role to locate the matching Organiser or Participant domain record.
- Event distance is not stored on Event; distance belongs to Race_Category because one Event can offer multiple race distances.
- Race_Category stores Category_Name, Distance_KM, Entry_Fee, Max_Participants and Current_Participants; Current_Participants is maintained by the system, not supplied by the client.
- Payment supports 0..* attempts per Enrollment, while business logic permits at most one successful Paid payment.

## Auth & Users

Register, log in, and manage the caller's own profile. JWT authentication carries the caller's user id and role.

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| POST | /api/auth/register | Registers a new Organiser or Participant. | None (public) | { role, organiserName?, firstName?, lastName?, dateOfBirth?, gender?, email, password, phoneNumber } | 201 Created - user id and role. 400 Bad Request - invalid role-specific data. 409 Conflict - email exists. |
| POST | /api/auth/login | Authenticates a user and returns a JWT containing user id and role. | None (public) | { email, password } | 200 OK - { token, role }. 401 Unauthorized - invalid credentials. |
| GET | /api/users/me | Returns the logged-in user's own profile. | Any logged-in user | None | 200 OK - caller profile. 401 Unauthorized - missing/invalid token. |
| PUT | /api/users/me | Updates the logged-in user's own profile. | Any logged-in user | Role-appropriate editable profile fields | 200 OK - updated profile. 400 Bad Request - invalid data. |

## Events

Organisers create, update and delete their own events. Both roles may view available events. Event distance is intentionally excluded because distance belongs to Race_Category.

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| GET | /api/events | Lists events, with optional date, type and location filters. | Any logged-in user | None | 200 OK - array of events. |
| GET | /api/events/{id} | Returns full details for one event. | Any logged-in user | None | 200 OK - event details. 404 Not Found. |
| POST | /api/events | Creates a new event owned by the logged-in Organiser. | Organiser | { eventName, description, eventDate, location, eventType, status?, closingDate } | 201 Created - new event. 400 Bad Request - validation error. |
| PUT | /api/events/{id} | Updates an event owned by the logged-in Organiser. | Organiser | { eventName?, description?, eventDate?, location?, eventType?, status?, closingDate? } | 200 OK - updated event. 403 Forbidden - not owner. 404 Not Found. |
| DELETE | /api/events/{id} | Deletes an event owned by the logged-in Organiser. | Organiser | None | 204 No Content. 403 Forbidden - not owner. 404 Not Found. |

## Race Categories

Each event may define one or more distance categories. Organisers manage categories; logged-in users may view them. Current_Participants is maintained by the system and is not client-editable.

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| GET | /api/events/{eventId}/categories | Lists the race categories available for an event. | Any logged-in user | None | 200 OK - array of categories. 404 Not Found - event does not exist. |
| POST | /api/events/{eventId}/categories | Adds a race category to an organiser's event. | Organiser | { categoryName, distanceKm, entryFee, maxParticipants } | 201 Created - new category. 400 Bad Request - invalid data. 403 Forbidden - not owner. |
| PUT | /api/categories/{id} | Updates a category belonging to the organiser's own event. | Organiser | { categoryName?, distanceKm?, entryFee?, maxParticipants? } | 200 OK - updated category. 400 Bad Request. 403 Forbidden. 404 Not Found. |
| DELETE | /api/categories/{id} | Removes a category from the organiser's own event. | Organiser | None | 204 No Content. 403 Forbidden. 404 Not Found. |

# Enrollment

Participants enter an event by selecting one of that event's Race_Categories. Enrollment stores Participant_ID and Category_ID; the Event is reached through Race_Category.Event_ID. Organisers may review enrollments for events they own.

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| POST | /api/events/{eventId}/enrolments | Enrolls the logged-in Participant into the selected category. | Participant | { categoryId } | 201 Created - enrollment with race number. 400 Bad Request - event closed/full or invalid category. 404 Not Found. 409 Conflict - duplicate enrollment. |
| GET | /api/enrolments/me | Lists the logged-in Participant's own enrollments. | Participant | None | 200 OK - array of own enrollments. |
| GET | /api/events/{eventId}/enrolments | Lists all enrollments for an event owned by the logged-in Organiser. | Organiser | None | 200 OK - array of enrollments. 403 Forbidden - not event owner. 404 Not Found. |

# Results

Organisers capture start time, finish time and finishing position after an event; participants may view their own results.

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| POST | /api/enrolments/{enrolmentId}/results | Creates the single result allowed for an enrollment. | Organiser | { startTime, finishTime, overallPosition, resultStatus? } | 201 Created - result record. 403 Forbidden - not event owner. 404 Not Found. 409 Conflict - result already exists. |
| PUT | /api/enrolments/{enrolmentId}/results | Corrects an existing result for an enrollment. | Organiser | { startTime?, finishTime?, overallPosition?, resultStatus? } | 200 OK - updated result. 403 Forbidden - not event owner. 404 Not Found. |
| GET | /api/results/me | Lists the logged-in Participant's own race results. | Participant | None | 200 OK - array of participant results. |
| GET | /api/events/{eventId}/results | Lists all results for an event as a leaderboard. | Any logged-in user | None | 200 OK - array of results. 404 Not Found - event does not exist. |

# Payments

Payment endpoints correspond to the Payment entity. An Enrollment may have multiple payment attempts, but only one successful Paid payment.

| HTTP Method | Route | Description | Role Required | Request Body | Expected Response |
|---|---|---|---|---|---|
| POST | /api/enrolments/{enrolmentId}/payments | Records a payment attempt for the logged-in Participant's enrollment. | Participant | { amount, paymentMethod } | 201 Created - payment attempt and status. 400 Bad Request - amount does not match Entry_Fee. 403 Forbidden - enrollment not yours. 409 Conflict - already paid. |
| GET | /api/enrolments/{enrolmentId}/payments | Returns payment attempts and current payment status for an enrollment. | Participant owner or event Organiser | None | 200 OK - payment history/status. 403 Forbidden - no ownership/access. 404 Not Found - no payment recorded. |

# Design Notes

**1. Role Enforcement** Middleware reads the JWT role claim (Organiser | Participant). Routes under /me always resolve to the authenticated user.

**2. User Profile** Both roles may view and update only their own profile information through /api/users/me.

**3. Authentication Mapping** After authentication, the API uses the user's unique email and role to locate the corresponding Organiser or Participant domain record.

**4. Event Ownership** Organisers may only update or delete events they created. Use the authenticated Organiser_ID in ownership queries.

**5. Event Information** Event stores Event_Name, Description, Event_Date, Location, Event_Type, Status and Closing_Date. Event_Type must be Run, Walk or Cycle. Distance is not stored on Event.

**6. Race Category Data** Race_Category stores Category_Name, Distance_KM, Entry_Fee, Max_Participants and Current_Participants. Distance_KM is required; Current_Participants is system-maintained.

**7. Category Ownership** Organisers may only create, update or delete categories belonging to events they own.

**8. Category Scoping** When enrolling, verify that the selected Category_ID belongs to the Event_ID in the route using Race_Category.Event_ID.

**9. Enrollment** Enrollment stores Participant_ID and Category_ID. The related Event is derived through Race_Category rather than duplicated in Enrollment.

**10. Duplicate Enrollments** UNIQUE (Participant_ID, Category_ID) prevents the same Participant entering the same category twice; return 409 Conflict.

**11. Category Capacity** Race_Category.Max_Participants caps entries. Current_Participants is maintained by the system; reject enrollment when the category is full.

**12. Event Status** Use Planned -> Open -> Closed -> Completed, with Cancelled available when required. Block enrollment after Closing_Date or when Closed/Completed/Cancelled.

**13. Results** Each Enrollment has at most one Race_Result. Organisers capture Start_Time, Finish_Time and Overall_Position. Completion time is derived rather than stored.

**14. Race Number Generation** Race_Number is generated automatically when a Participant successfully enrolls and must be unique for every Enrollment.

**15. Payments** Payment.Enrollment_ID is not unique because multiple attempts are allowed. Payment.Amount must equal the category Entry_Fee, and only one successful Paid payment is permitted.

**16. Part 2 Alignment** Implement the API to match this plan and the final UML ERD/SQL Server schema. Avoid adding unplanned entities without updating all three documents.

**Recommended build order:** Auth + Events + Categories + Enrollment first, then Results + Payments.